

SMART INDIA HACKATHON 2026

PROBLEM STATEMENT 26132 | GOVERNMENT OF MAHARASHTRA

MANDI-SETU

Build Order

6 people | 3 days | 6 lanes that do not collide

Target: 80-90% for the college round. The rest before SIH.

Team

Akash - Nikhil - Kartik - Nilesh - Pranay - Shreya

Six members. Shreya owns the farmer front-end (Lane 4).

Window

Friday to Sunday build. Pitch Monday to Tuesday.

How to use it

Clone the repo. Pick a lane. Paste that lane's boot prompt into Claude Code. It reads the repo and starts Task 1.

THE ONLY RULE THAT MATTERS

You may edit only the files your lane owns. Need a change elsewhere?
Append to docs/BLOCKERS.md, stub locally, keep moving.

Plan document. No code in here by design - the code lives in the repo.

00 How to Use This Document

This is the document you hand to your teammates. It answers four questions and nothing else: where the project stands right now, what the six lanes are, what each lane does hour by hour, and how a teammate turns this page into a working Claude Code session.

It deliberately contains no code. The code lives in the repository, and the repository is the source of truth. This page is the index.

The three-step onboarding, exactly

1. ****Clone and boot the machine.**** Five commands, listed in section 04. Ten minutes if the network behaves.
2. ****Pick a lane**** from the table in section 03. Six lanes, one each. Shreya takes Lane 4; the other five are open and you can trade among yourselves in the first ten minutes.
3. ****Open Claude Code in the repo root and paste your lane's boot prompt**** from section 06. It reads `CLAUDE.md` automatically, then reads your lane brief, then starts Task 1 and tells you its plan before writing files.

Why the boot prompt is a path, not a wall of text

Claude Code reads the repository. A pasted PDF is a snapshot it cannot re-read, cannot grep, and cannot check against the current state of the code. So each boot prompt names the files to read - CLAUDE.md, your lane brief, the contract - and the agent reads them itself. This document is for the humans on the team. The markdown in docs/ is for the agents.

What is already written, and what this adds

Document	What it is	Read it when
`CLAUDE.md`	Repository law. Ten invariants, the ownership map, coding standards. Claude Code loads it automatically on every session.	Before your first edit. All of it.
`docs/00_MASTER_BUILD_PLANN.md`	Architecture, the request pipeline every route follows, the 72-hour timeline, the risk register.	Section 2.3 before your first API route.
`docs/01_CONTRACTS.md`	The API surface. Frozen. Both sides build against it.	Whenever you touch a request or response shape.
`docs/02_SECURITY_AND_QUALITY.md`	The S-rules and the pre-demo security pass.	Lane 1 reads all of it; everyone reads the rules tagged with their lane.
`docs/03_DEMO_AND_SEED.md`	The frozen 12-beat demo script with the exact numbers, and the seed specification.	Before you build any screen. The numbers in it are contractual.
`docs/roles/R1..R5_*.md`	Per-lane task lists with acceptance criteria and named failure modes.	Your own. Continuously. Do not read the others.
This document	The 6-lane split, the compressed 3-day calendar, the cut list, and the onboarding mechanism.	Now, once, all of it. Then keep section 05 open.

01 Where We Are Right Now

This section is a verified inventory, not an estimate. Every line below was checked against the working tree on 4 September 2026.

Committed and on disk

Area	State	Lines
`prisma/schema.prisma`	Complete. ~30 models. Integer paise, integer kilograms, basis points throughout. Includes `baselineMethod` on the ledger and `SELL_ELSEWHERE` in the action enum.	716
`packages/contracts/src/index.ts`	Complete. Zod schemas for every endpoint. Includes `PledgeQuote`, `worstCasePaisePerQtl`, `cashNeedBy`.	645
`packages/contracts/src/fixtures.ts`	Complete. Contract-shaped fixtures so the front-end lanes build before the back-end lanes finish.	276
`docs/` (11 files)	Complete. Build plan, contracts, security, demo script and seed spec, gap review, pitch plan, data sources, blocker log.	~2,300
`docs/roles/` (5 briefs)	Complete. Task lists with acceptance criteria.	~1,850
`prisma/seed/rng.ts`	Complete. Deterministic mulberry32 with named sub-streams. No `Math.random()` anywhere in the seed.	94
`prisma/seed/index.ts`	Skeleton. Run order fixed, reset guard written, the eight demo-protecting assertions documented as TODOs.	115
`services/ml/app/contracts.py`	Complete. Pydantic mirror of the Zod contract.	184
`package.json`, `docker-compose.yml`, `.env.example`	Complete. Workspace scripts, Postgres on port 5433, environment template with no secrets.	87

Not written yet - and this is the honest part

There is no application code in this repository. None.

apps/web/ does not exist. No Next.js app, no route handlers, no components, no domain functions, no ML service beyond its contract file. node_modules is not installed. The database has never been created. What exists is a complete and internally consistent specification, plus the schema and the contracts it is built on.

That is a better position than it sounds, and worse than it looks. Better, because the expensive decisions - the data model, the API surface, the demo numbers, the ownership boundaries - are made and frozen, which is exactly the work that six parallel agents cannot do concurrently. Worse, because every line of the product itself is still ahead of us and the first ninety minutes are single-threaded on one person.

The single-threaded hour, named

Lane 1 must scaffold `apps/web` before Lanes 3, 4 and 6 have a directory to write into. Until that lands, three people cannot typecheck anything. Two mitigations, both in the calendar:

- ****Lane 1 ships the scaffold as commit one, inside 90 minutes****, before touching auth. Not a perfect scaffold - `next dev` serving a blank page with Tailwind and the contracts package resolving is enough.
- ****Lanes 2 and 5 are not blocked at all.**** The Python service and the seed files live outside `apps/web`. They start real work at minute zero and should be the first two commits on the board after the scaffold.
- ****Lanes 3, 4 and 6 spend that window on setup and reading****, which genuinely takes ninety minutes: clone, install, Postgres up, `.env`, read `CLAUDE.md` and their brief, create their branch, boot their agent, agree their first three files. Nobody is idle; nobody is compiling either.

Open items carried in from the last session

ID	What	Owner	When
B5	`Recommendation` needs `cashNeedPaise` and `pledgeOffered` audit columns so a pledge quote we showed can be reconstructed later.	Lane 1	Before the contract freeze
B7	`BuyerDto` needs `renegotiationBps` - the buyer-reliability number the pitch leans on.	Lane 1	Before the contract freeze
V1	`npm run typecheck` has never been run. The contract and schema changes are verified by reading, not by the compiler.	Lane 1	First hour, right after `npm i`
G1	The repo is committed locally but not pushed . One command, section 12.	Pranay	Today

02 What "80-90% Complete" Means

Do not measure completeness in features, and do not measure it in commits. Measure it against this list. If all ten are demonstrably true on Monday morning, you are at 90% and the presentation writes itself. If eight are true and the two missing ones are named honestly on a roadmap slide, you are at 80% and still in a strong position.

#	Statement that must be true on stage	Lanes
C1	A farmer logs in with a phone number and an OTP, in Marathi, on a 360 px screen.	1 + 4
C2	He sees today's price at his nearest markets **net of transport** , not gross. The reordering is visible.	2 + 4
C3	He gets a p10/p50/p90 forecast with a model card showing MASE against seasonal-naive and empirical coverage.	2
C4	He asks "should I sell?" and gets SELL_NOW / HOLD / SPLIT with a rupee figure, an itemised cost breakdown, **and the worst case at the same type size** .	2 + 4
C5	On a volatile crop the same screen returns **NO_ADVICE** with a written reason, driven by real band width and not a flag.	2 + 4
C6	He lists a lot, self-grades it through a 6-question assay, and gets a grade plus a weakest-dimension tip.	3 + 4
C7	Three farmers pool into one truckload, each consents to a grade-weighted split **before** the sale, and each sees his value versus selling alone.	3 + 4
C8	A buyer sees matched lots with a why-this-match sentence, sends an offer, farmer accepts, escrow funds, transit, delivered, QC, released.	3 + 6
C9	The ledger shows a **measured** rupees-per-quintal gain against the same-day mandi benchmark, and the hash chain verifies. Edit one row and it names the broken sequence number.	3
C10	`npm run verify` passes - typecheck, lint, unit tests, production build - and the app is on a public URL.	1

C4, C5 and C9 are the three that win it.

A rupee figure with its downside next to it. A model that refuses to answer when it should not answer. A gain that is computed by subtraction and can be verified live. If you are out of time, those three survive and everything else is negotiable. Section 07 is the pre-agreed order in which everything else goes.

The hero moment, so every lane knows what it is serving

One farmer, one screen, eleven days. The window screen returns HOLD with an expected gain of Rs. 1,428 on twelve quintals of onion, net of storage, spoilage and interest, with the worst case of minus Rs. 480 printed in the same size beside it. Then the farmer taps GET MONEY TODAY and receives a pledge quote against a WDRA warehouse receipt: Rs. 34,000 in hand today, Rs. 370 of interest, repaid from the sale. He gets the cash and keeps the upside.

And the guardrail is in the code, not in the pitch: if the interest ever exceeds the expected gain, no loan is offered at all. A system that always offers you credit is a moneylender. That sentence is the reason the feature is defensible, and it is why every lane needs to know that beat 7 of `docs/03\_DEMO\_AND\_SEED.md` is the peak of the pitch.

03 The Six Lanes

Six lanes, six people, no shared files. The lane names match the role briefs already in `docs/roles/`, so your agent has a task list the moment you pick one. Lane 6 is new: it takes the buyer, FPO and admin consoles that were previously bundled into Lane 5.

Lane	Codename	Owns	Suggested
**L1*	SPINE	Scaffold, Postgres, auth, `guard()`, contracts, deploy, integration duty. Unblocks everyone and says no at the freeze.	Pranay
**L2*	ORACLE	Python service, price ingestion, quantile forecast, backtest, the sale-window optimiser and the pledge quote. The differentiator.	Akash
**L3*	LEDGER	Lots, grading, fair split, matching, offers, the escrow state machine, the hash-chained ledger, disputes.	Nikhil
L4*	KISAN	The farmer PWA. Marathi-first, 360 px, the window screen, the refusal screen, the audio button. **Shreya.	Shreya
**L5*	BAZAAR	The seed dataset, the two demo personas, determinism, E2E tests, rehearsal. Everyone consumes this.	Kartik
**L6*	CONSOLES	Buyer console, FPO console, admin data-quality console. Plus the deck and the official SIH Idea PPT.	Nilesh

The suggested column is a suggestion. Trade in the first ten minutes, then stop.

Two constraints only. Shreya takes Lane 4, as agreed. And Lane 2 needs whoever is most comfortable with Python, pandas and a command line - that lane fails on environment problems more than on modelling. Everything else is genuinely interchangeable. What is not negotiable is that the trading stops at minute ten and nobody touches another lane's files afterwards.

Why Lane 6 is consoles plus the deck

It looks like an odd pairing and it is the deliberate one. The consoles are the most cuttable code in the project - they sit at positions two and four in the drop list - while the deck is the least cuttable deliverable of all, because the college round is a presentation and not a code review. Pairing the most droppable code with the least droppable artefact means that when time runs out, Lane 6 sheds console screens and the deck still gets finished. Pair the deck with a critical-path lane instead and you end up choosing between a working demo and a deck, at midnight, which is not a choice anyone makes well.

Ownership map - the file boundaries, verbatim

This is the collision-prevention mechanism and it is the only one. Read your row. Do not edit outside it, not even a one-line fix, not even if it is obviously wrong.

Lane	Paths you may create, edit and delete
L1	`apps/web/src/lib/*.ts` (db, auth, guard, money, http, env, audit, idem, ml, ratelimit) - `apps/web/app/api/auth/**` - `app/api/ref/**` - `apps/web/src/components/ui/**` - `apps/web/middleware.ts` - `prisma/schema.prisma` - `prisma/migrations/**` - `prisma/seed/00_reference.ts` - `packages/contracts/**` - root config, CI, `docker-compose.yml`, `.env.example` - `CLAUDE.md`, `docs/00_*`, `docs/01_*`, `docs/02_*`
L2	`services/ml/**` (the entire Python service) - `apps/web/src/lib/domain/window.ts` - `costs.ts` - `apps/web/src/lib/ml.ts` - `apps/web/app/api/prices/**` - `app/api/window/**` - `prisma/seed/10_prices.ts` - `docs/roles/R2_ORACLE.md`
L3	`apps/web/src/lib/domain/{grading,matching,escrow,ledger,split}.ts` - `apps/web/src/lib/domain/__tests__/**` - `apps/web/app/api/{lots,pools,demand,match,offers,tx,ledger,disputes}/**` - `docs/roles/R3_LEDGER.md`
L4	`apps/web/app/(farmer)/**` - `apps/web/src/components/farmer/**` - `apps/web/src/components/charts/**` - `apps/web/messages/{en,mr}.json` - `apps/web/public/audio/**` - `docs/roles/R4_KISAN.md`
L5	`prisma/seed/**` except `00_reference.ts` and `10_prices.ts` - `tests/e2e/**` - `docs/03_DEMO_AND_SEED.md` - `docs/deck/SOURCES.md` - `docs/roles/R5_BAZAAR.md`

Lane	Paths you may create, edit and delete
L6	`apps/web/app/(buyer)/**` - `app/(fpo)/**` - `app/(admin)/**` - `apps/web/src/components/buyer/**` - `docs/deck/**` except `SOURCES.md` - `docs/roles/R6_CONSOLES.md`
all	`docs/BLOCKERS.md` - append only, never rewrite or delete another entry.

Anything not listed above belongs to Lane 1. Two paths deserve a specific warning because they look shared and are not: ``apps/web/src/components/ui/**`` is Lane 1's shadcn primitives, installed once - compose them, never modify them; and ``apps/web/messages/{en,mr}.json`` is Lane 4's, so if you need a string you request it rather than adding a key yourself.

04 Booting Your Machine and Your Agent

Step 1 - the repository, once per person

ONCE, ON EVERY MACHINE

```
git clone https://github.com/Zakuroooo/Oneforall-Farmers.git
cd Oneforall-Farmers
npm install
npm run db:up          # Postgres in Docker, on port 5433
cp .env.example .env
openssl rand -base64 48 # paste into AUTH_SECRET in .env
```

Then, and only after Lane 1 has pushed the scaffold and the schema:

AFTER THE SCAFFOLD LANDS

```
npm run db:push          # create the tables
npm run db:seed          # deterministic demo data
npm run dev              # http://localhost:3000
```

Lane 2 additionally, and the Python version is not a preference:

LANE 2 ONLY

```
cd services/ml
uv venv --python 3.11      # NOT python3. The default is 3.14 and
                           # no LightGBM wheel exists for it.
uv pip install -r requirements.txt
.venv/bin/uvicorn app.main:app --reload --port 8000
```

Step 2 - your branch, once, and you stay on it

BRANCH DISCIPLINE

```
git checkout -b l4-kisan    # substitute your own lane

# every 30 to 45 minutes, all three hours:
git add -A && git commit -m "feat(kisan): window screen, worst-case row"
git push

# at every sync window, before writing another line:
git fetch origin && git rebase origin/main
```

Rebase, never merge. And an unpushed branch does not exist.

Six branches repeatedly merging main into themselves produces a history where git blame is useless and the same conflict resurfaces at every sync. Rebase keeps it linear and resolves each conflict once. If you hit a conflict in a file you do not own: abort the rebase, take theirs, re-apply your own change. Do not fix their file. And push every commit - a laptop that dies with four hours of unpushed work has cost the team those four hours twice.

Step 3 - boot your agent

Open Claude Code in the repository root. It loads `CLAUDE.md` by itself. Paste the boot prompt for your lane from section 06. Every boot prompt has the same five parts, and the reason each part is there is worth knowing, because you will be re-pasting it at the start of every session for three days:

Part	What it does	Why it is in there
Project context	One paragraph: what MANDI-SETU is, that six agents are working in one repo.	Without it the agent optimises for a solo codebase and proposes repo-wide refactors.
Files to read, in order	`CLAUDE.md`, the build plan, the contract, your lane brief.	The agent reads the repo. Naming paths beats pasting text it cannot re-read.

Part	What it does	Why it is in there
Paths you own	The explicit list from section 03.	Agents are helpful by default and will happily fix a file in someone else's directory.
The blocker rule	Append to `docs/BLOCKERS.md`, stub locally, keep moving.	Turns a 40-minute stall into a 2-minute note. This is the single highest-value line.
Start Task 1, plan first	"Tell me your plan before you write files."	Catches a misread task before it becomes six files you have to unpick.

05 The Three-Day Calendar

The H-numbers in docs/ assume 72 hours. You have about 54, and roughly 36 of them are working hours per person.

Every role brief in docs/roles/ is written on a 72-hour scale, H0 to H72. That scale was built for a 3-day hackathon with two sleep blocks. Your window is Friday afternoon to Monday morning. Do not try to reconcile the numbers in your head at hour 40 - use the compression table below, which maps every milestone the briefs reference onto a wall-clock time. Where they disagree, this table wins.

The compression table

Assuming a Friday 15:00 start. Shift every row by the same amount if you start later, and keep the ordering intact - the ordering is what the dependencies need.

Brief says	Milestone	Do it by	What must be true
H0-H4	Joint lockdown. All six in one room. Nobody codes alone.	Fri 15:00-19:00	Contracts signed off and frozen. B5 and B7 resolved. Everyone booted, branched, one trivial commit pushed each.
H4	**CONTRACT FREEZE**	Fri 19:00	`packages/contracts` and `prisma/schema.prisma` stop changing. Additive only, via a blocker entry and Lane 1's approval.
H12	Data gate	Sat 03:00 or first thing Sat	Lane 2 has real price data on disk. Not a model - data. Lane 5 has reference rows seeded.
H20	**CHECKPOINT A** - walking skeleton	Sat 12:00	Login works. Prices screen renders from seeded data. Escrow FSM passes its unit tests. Warehouses exist. Both personas resolve to their intended verdicts.
H36	Hero and audio	Sat 23:00	Window screen renders HOLD with the worst-case row and the pledge card. Marathi audio files exist or a teammate has recorded them.
H48	**GOLDEN PATH COMPLETE**	Sun 12:00	The full chain walks end to end: list, assay, pool, consent, match, offer, accept, fund, dispatch, deliver, QC, release, ledger, verify.
H58	**FEATURE FREEZE**	Sun 20:00	Nothing new. Bugfixes, copy and rehearsal only. Lane 1 enforces this and is expected to be unpopular for an hour.
H64	Security and quality pass	Sun 23:00	Part C of `docs/02_SECURITY_AND_QUALITY.md` run personally. `npm run verify` green. Deployed.
H68	Rehearsal done	Mon 07:00	Three timed run-throughs under 7:00. Fallback video cut. Both drivers know every tap.
-	**PITCH**	Mon-Tue	Wifi off, on stage, visibly.

Day 1 - Friday. Foundations, and the scaffold is the bottleneck

Lane	Friday
L1	Scaffold `apps/web` and push it inside 90 minutes - this unblocks three people. Then Postgres up, `db:push`, `npm run typecheck` for the first time, resolve B5 and B7, freeze the contract at 19:00. Then start auth: OTP request, OTP verify, JWT in an httpOnly cookie.
L2	Environment first, with Python 3.11. Then ingestion: get real Maharashtra price data onto disk and write `seed_prices.csv`. Your gate tonight is **data** , not a model. Seasonal-naive baseline only if data is done early.
L3	`escrow.ts` before any route. Sixty lines, a transition table as data, and the unit tests in the same commit - every legal transition passes, every illegal one throws, terminals are terminal. Then `POST /api/lots` and `GET /api/lots` with the owner in the `where` clause.
L4	i18n before any screen - the JSON dictionaries and the language context. Then the app shell with a four-item bottom nav at 44 px, the login screen, and home. Build against `@mandi/contracts/fixtures`, not against Lane 2.

Lane	Friday
L5	Reference data: 7 districts, 8 to 10 real APMC markets with Lasalgaon mandatory, 6 commodities. Then **3 to 5 warehouses with at least 2 WDRA-registered ones in the Nashik belt** - without those rows the pledge quote is null for every lot and beat 7 of the demo silently disappears. Then the two personas.
L6	Read the brief, boot, branch. Then the buyer console shell and the demand-posting form against fixtures. Start the deck outline tonight - slide titles and the punch list of screenshots you will need, with a due date on each.

Day 2 - Saturday. The hero, and the chain

Lane	Saturday
L1	Finish auth. <code>`guard()`</code> - the single most security-critical function in the repo, returning 404 and not 403 for rows an actor may not see. <code>`http.ts`</code> envelope. Reference endpoints. Deploy to a public URL today, not on Sunday. From noon you are half-time on integration duty.
L2	Features, then LightGBM quantile regression at alpha 0.1, 0.5 and 0.9, then the backtest that produces the MASE and coverage numbers for the deck. Then <code>`costs.ts`</code> with real Maharashtra numbers and citations in comments. Then the optimiser and <code>`window.ts`</code> . **Then the pledge quote** - it is the moat and it must not become a slide.
L3	Assay, list, pool, consent - and the pool cannot leave <code>CONSENT_PENDING</code> until every member has an explicit consent row, enforced in the route and not in the UI. Then matching with visible score components and a why sentence. Then offers with idempotency. Then the transition route writing status, escrow event, audit row and ledger append in one transaction.
L4	**The window screen.** HOLD verdict, the rupee figure, the itemised costs, the worst-case row at the same type size, the pledge card, the listen button. Then the refusal screen, designed as carefully as the hero. Then the lot wizard and the pool consent sheet. Marathi audio done by 23:00 - if the TTS is not working, record a Marathi-speaking teammate and label it honestly.
L5	The transaction history file - about 200 transactions over six months with a valid hash chain and **15 to 20% of ledger rows negative**, because a ledger where every farmer won is obviously fabricated. Then verify the citations, ruthlessly, and record them in <code>`docs/deck/SOURCES.md`</code> .
L6	Buyer console: matched lots showing the why sentence and the buyer reliability numbers including renegotiation rate. FPO console: member lots, pool builder, split table with weights visible. Deck: two slides per sync window, with real screenshots as they land.

Day 3 - Sunday. Harden, freeze, rehearse

Lane	Sunday
L1	Integration full-time until the freeze. <code>`audit.ts`</code> , <code>`idem.ts`</code> , <code>`ratelimit.ts`</code> , <code>`middleware.ts`</code> if not already done. Then Part C of the security document, personally, every line. <code>`npm run verify`</code> green. Production deploy. Phone test over mobile data. **Then enforce the 20:00 feature freeze.**
L2	<code>`eval.py`</code> prints the backtest table for the deck - one command, one table, real numbers. Unit tests on <code>`window.ts`</code> : HOLD when the gain is real, <code>SELL_NOW</code> below the threshold, <code>NO_ADVICE</code> on a wide band, and no pledge quote when the interest exceeds the gain.
L3	Unit tests complete. Then walk the full chain yourself **ten times** - the eighth walk is where you find the state that leaves a transaction stuck. There must be no state from which a transaction cannot reach a terminal state, and you prove that by walking it, not by reading the table.
L4	Three states on every screen - loading skeletons, empty states with a next action, error states a farmer can act on. Then the Marathi overflow pass at 360 px, every screen, because Marathi strings run about 30% longer than English. Then contrast and tap targets.
L5	Verify all ten checkpoints from section 02 personally. Cross-lane bug bash - you walk Lane 4's farmer flow, Lane 4 walks yours, and you both file what you find. Then three timed rehearsals and the fallback video, cut before the freeze and not after it.
L6	Admin data-quality console - coverage per mandi, the imputed-share badge. Unusual, and it lands, because showing your own data quality before a judge asks is the whole trust argument in one screen. Then finish both decks: the official SIH Idea PPT mapped 1:1 to the template, and the 7-minute college deck.

06 The Six Lane Briefs and Their Boot Prompts

One page per lane. Read only your own. The boot prompt is what you paste into Claude Code at the start of every session - not just the first one, because a fresh session has no memory of the ownership rule and will cheerfully edit someone else's file without it.

L1 SPINE - suggested: Pranay

Mission. Nobody else can move if the skeleton is not standing. Ship the scaffold, the schema, auth and the contracts before you ship anything clever.

Order of work. The scaffold in 90 minutes, then postgres, then auth end to end, then guard(), then deploy on day one.

Your brief. `docs/roles/R1\_SPINE.md`

Six things that decide whether this lane succeeds:

- Scaffold `apps/web` and push it first. Three lanes are blocked until you do.
- Resolve B5 and B7 from `docs/BLOCKERS.md`, then freeze the contract at Friday 19:00.
- Run `npm run typecheck` before anything else - it has never been run against the contracts.
- `guard()` is the most security-critical function in the repo. Owner in every `where`. 404, never 403.
- Deploy on Saturday, not Sunday. A first deploy on the last night is how demos are lost.
- You enforce the Sunday 20:00 feature freeze. Somebody will push back. Hold it anyway.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L1 (codename SPINE) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R1_SPINE.md (my task list, in order)
6. docs/02_SECURITY_AND_QUALITY.md (all of it - it is mine)

I OWN and may edit ONLY these paths:

apps/web/src/lib/*.ts (db, auth, guard, money, http, env, audit, idem, ml)
apps/web/app/api/auth/** apps/web/app/api/ref/**
apps/web/src/components/ui/** apps/web/middleware.ts
prisma/schema.prisma prisma/migrations/** prisma/seed/00_reference.ts
packages/contracts/** root config, CI, docker-compose.yml, .env.example
CLAUDE.md docs/00.* docs/01.* docs/02.*

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L1): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- Scaffold apps/web FIRST and push it. Three other lanes are blocked until it lands.
- guard() returns 404, never 403, for rows the actor may not see. Owner in every where.
- Money is integer paise. formatPaise() at the render edge is the only formatter.
- env.ts fails loudly at boot, once, listing every missing variable at the same time.
- No secrets in git. Only .env.example. If one lands, tell me immediately and rotate it.
- The contract freezes Friday 19:00. After that, additive-only via a BLOCKERS entry.

After each task run `npm run typecheck && npm run test`, then commit (feat(spine): ...) and push. Commit every 30-45 minutes.

Start with T1.1 in docs/roles/R1_SPINE.md and tell me your plan before you write any files.

L2 ORACLE - suggested: Akash

Mission. A measured MASE of 0.91 with an honest caveat beats a claimed 95% accuracy every time. Your NO_ADVICE

path is a feature, not a fallback.

****Order of work.**** Python 3.11 environment, then real data on disk, then a baseline, then lightgbm, then the optimiser, then the pledge quote.

****Your brief.**** `docs/roles/R2\_ORACLE.md`

****Six things that decide whether this lane succeeds:****

- `uv venv --python 3.11`. Not `python3`. The default is 3.14 and no LightGBM wheel exists for it.
- Your Friday gate is ****data on disk****, not a model. A model with no data is fiction with a chart.
- p10/p50/p90 or nothing. A point prediction with no interval is a guess.
- Refuse when the band is too wide, and put a real Marathi sentence in `refusalReason`.
- The pledge quote is computed in TypeScript. Python returns null for it, always.
- The guardrail is code: no quote when interest exceeds the expected gain. Unit-test that.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L2 (codename ORACLE) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R2_ORACLE.md (my task list, in order)
6. prisma/schema.prisma and packages/contracts/src/index.ts (read-only for me)

I OWN and may edit ONLY these paths:

services/ml/**	(the entire Python service)
apps/web/src/lib/domain/window.ts	apps/web/src/lib/domain/costs.ts
apps/web/src/lib/ml.ts	
apps/web/app/api/prices/**	apps/web/app/api/window/**
prisma/seed/10_prices.ts	docs/roles/R2_ORACLE.md

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L2): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- Python 3.11 via `uv venv --python 3.11`. NOT python3 - default 3.14 has no LightGBM wheel.
 - My gate on day one is DATA ON DISK, not a model. Baseline before LightGBM.
 - Always p10/p50/p90. Never a bare point prediction.
 - When the band exceeds the threshold, return NO_ADVICE with a real Marathi sentence.
 - The pledge quote is computed in TypeScript (costs.ts computes, window.ts decides). Python returns null for it, always.
 - Guardrail in code: emit no pledge quote when interest exceeds the expected gain.
 - Money is integer paise. The domain layer is pure - no db, no fetch, no Date.now().
- After each task run `npm run typecheck && npm run test`, then commit (feat(oracle): ...) and push. Commit every 30-45 minutes.

Start with T2.1 in docs/roles/R2_ORACLE.md and tell me your plan before you write any files.

L3 LEDGER - suggested: Nikhil

****Mission.**** You own the number on the winning slide. It must be measured by subtraction, never asserted, never hardcoded.

****Order of work.**** Escrow.ts and its tests before any route, then lots, then grading and split, then matching, offers and the ledger.

****Your brief.**** `docs/roles/R3\_LEDGER.md`

****Six things that decide whether this lane succeeds:****

- `escrow.ts` first. Sixty lines, a table as data, tests in the same commit.
- A TxStatus changes in exactly one file. No route writes `status` directly.
- Integer paise. Split N ways by integer division, remainder to the largest share, shares sum exactly.

- Every read scoped to the actor. `findFirst` with the owner in the `where`, never `findUnique` on a body id.
- `deltaPaise` computed by subtraction. `baselineMethod` on the row names the counterfactual.
- Walk the full chain ten times on Sunday. The eighth walk finds the stuck state.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L3 (codename LEDGER) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R3_LEDGER.md (my task list, in order)
6. docs/02_SECURITY_AND_QUALITY.md (S1, S3, S9, S10, S11, S12 are mine)
7. prisma/schema.prisma and packages/contracts/src/index.ts (read-only for me)

I OWN and may edit ONLY these paths:

```
apps/web/src/lib/domain/{grading,matching,escrow,ledger,split}.ts
apps/web/src/lib/domain/__tests__/**
apps/web/app/api/{lots,pools,demand,match,offers,tx,ledger,disputes}/**
docs/roles/R3_LEDGER.md
```

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L3): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- escrow.ts FIRST, before any route, with its tests in the same commit.
- A TxStatus may change ONLY inside escrow.ts. Illegal transitions throw.
- realisation_ledger and audit_log are APPEND-ONLY. Never update, never delete.
- Money is integer paise. Split N ways: integer-divide, remainder to the largest share, shares sum EXACTLY to the total.
- Every read scoped by the session actor: findFirst with the owner in the where, never findUnique on an id from the request body. 404, not 403.
- deltaPaise computed by subtraction, never supplied. baselineMethod on every row.
- Every state-creating write is idempotent via the x-idempotency-key header.
- A pool cannot leave CONSENT_PENDING until every member has an explicit consent row.

After each task run `npm run typecheck && npm run test`, then commit (feat(ledger): ...) and push. Commit every 30-45 minutes.

Start with T3.1 in docs/roles/R3_LEDGER.md and tell me your plan before you write any files.

L4 KISAN - suggested: Shreya

Mission. The judge looks at your screens for the entire seven minutes. Marathi first, thumb-reachable, and a farmer must never see a stack trace.

Order of work. I18n before any screen, then the shell and login, then prices, then the window screen, then the refusal screen.

Your brief. `docs/roles/R4\_KISAN.md`

Six things that decide whether this lane succeeds:

- i18n first, before a single screen. This is the decision that saves you on Sunday.
- Build against `@mandi/contracts/fixtures`. Do not wait for Lane 2 or Lane 3.
- 360 px, 44 px tap targets, everything in the thumb zone. Test at 360 px, not at 1440.
- The worst-case row is the **same type size** as the gain. Not small print. That is the whole ethic.
- Marathi audio by Saturday 23:00. If TTS fails, record a teammate and label it honestly.
- Marathi runs ~30% longer than English. Set the locale to Marathi and walk every screen.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L4 (codename KISAN) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R4_KISAN.md (my task list, in order)
6. packages/contracts/src/fixtures.ts (my back end until Sunday - read it fully)

I OWN and may edit ONLY these paths:

```
apps/web/app/(farmer)/**
apps/web/src/components/farmer/**      apps/web/src/components/charts/**
apps/web/messages/en.json              apps/web/messages/mr.json
apps/web/public/audio/**                docs/roles/R4_KISAN.md
```

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L4): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- i18n FIRST, before any screen. Marathi is the default, English is the option.
- Build against @mandi/contracts/fixtures. Never wait for another lane.
- 360 px viewport, 44 px minimum tap targets, primary actions in the thumb zone.
- The worst-case number is the SAME type size as the gain. Never small print.
- Server components by default. "use client" only for state, effects or handlers.
- Every screen ships loading, empty and error states - not just the happy path.
- Marathi strings run ~30% longer than English. Test in Marathi first.
- I own apps/web/src/components/ui/** as READ-ONLY: compose the primitives, never modify them. They are L1's.

After each task run `npm run typecheck && npm run test`, then commit (feat(kisan): ...) and push. Commit every 30-45 minutes.

Start with T4.1 in docs/roles/R4_KISAN.md and tell me your plan before you write any files.

L5 BAZAAR - suggested: Kartik

Mission. You own the story. If the seed data is not believable, nothing else in the demo matters.

Order of work. Reference data and warehouses first, then the personas, then prices, then history, then verification and rehearsal.

Your brief. `docs/roles/R5\_BAZAAR.md`

Six things that decide whether this lane succeeds:

- Everything deterministic. `rngFor(name)` only. No `Math.random()`, no `new Date()` in generated history.
- Two `db:reset` runs must produce byte-identical ledger totals. That is the acceptance test.
- **At least 2 WDRA warehouses in the Nashik belt.** Without them beat 7 disappears silently.
- Both personas verified after every reset. Sunita's refusal comes from real band width, never a flag.
- 15 to 20% of ledger rows negative. A ledger where every farmer won is obviously fake.
- No Aadhaar numbers anywhere, not even fake ones. Phone is the identifier.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L5 (codename BAZAAR) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R5_BAZAAR.md (my task list, in order)
6. prisma/seed/rng.ts (already written - every random number comes from here)

I OWN and may edit ONLY these paths:

```
prisma/seed/**    EXCEPT 00_reference.ts (L1) and 10_prices.ts (L2)
tests/e2e/**
docs/03_DEMO_AND_SEED.md    docs/deck/SOURCES.md
docs/roles/R5_BAZAAR.md
```

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L5): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- Everything deterministic. rngFor(name) only. No Math.random() anywhere under prisma/seed. No new Date() in generated history - derive from SEED_TODAY.
 - Acceptance: two `npm run db:reset` runs produce byte-identical ledger totals.
 - At least 2 warehouses with isWdra = true in the Nashik belt. Without them the pledge quote is null for every lot and demo beat 7 disappears silently.
 - Both personas re-verified after every reset: Ramesh gets HOLD with a pledge quote, Sunita gets NO_ADVICE from real band width and never from a flag.
 - 15 to 20 percent of ledger rows have deltaPaise < 0. A ledger where every farmer won is obviously fabricated.
 - Anything generated carries source = SYNTHETIC and is badged in the UI.
 - No Aadhaar numbers anywhere, not even fake ones. Phone is the identifier.
- After each task run `npm run typecheck && npm run test`, then commit (feat(bazaar): ...) and push. Commit every 30-45 minutes.

Start with T5.3 in docs/roles/R5_BAZAAR.md (T5.1 and T5.2 are already done) and tell me your plan before you write any files.

L6 CONSOLES - suggested: Nilesh

Mission. The consoles prove this is a two-sided market and not a farmer app with a story. The deck is what the college round actually judges.

Order of work. Buyer console shell and demand form, then matched lots with the why sentence, then fpo, then admin, and the deck from hour zero.

Your brief. `docs/roles/R6\_CONSOLES.md` (write it from this section first)

Six things that decide whether this lane succeeds:

- Your lane brief does not exist yet. Write `docs/roles/R6\_CONSOLES.md` from this section as your first commit.
- Buyer console shows **renegotiation rate**, not just a star rating. That number is the trust argument.
- FPO split table shows the actual `weight` per member. Auditable, on screen.
- The admin data-quality console is unusual and it lands. Coverage per mandi, imputed share labelled.
- The deck starts now and gets ten minutes at every sync window. Never after the freeze.
- Two artefacts, not one: the official SIH Idea PPT mapped 1:1 to the template, and the 7-minute deck.

PASTE THIS INTO CLAUDE CODE, EVERY SESSION

I am L6 (codename CONSOLES) on MANDI-SETU, a Next.js 15 + Prisma + Postgres monorepo for Smart India Hackathon 2026, problem statement 26132 (market linkages and price discovery for farmers of Maharashtra). SIX engineers with six Claude Code agents are building this in ONE repo over three days.

Read these before you write anything, in this order:

1. CLAUDE.md (ten binding invariants + ownership map)
2. docs/00_MASTER_BUILD_PLAN.md (section 2.3 is the route pattern)
3. docs/01_CONTRACTS.md (frozen API surface)
4. docs/03_DEMO_AND_SEED.md (the numbers on screen are contractual)
5. docs/roles/R5_BAZAAR.md sections T5.7, T5.8, T5.9 and T5.13 (my tasks, carved out of R5)
6. packages/contracts/src/fixtures.ts (my back end until Sunday)
7. docs/11_PITCH_PPT_AND_SELECTION_ROUNDS.md (both decks are mine)

I OWN and may edit ONLY these paths:

```
apps/web/app/(buyer)/**      apps/web/app/(fpo)/**      apps/web/app/(admin)/**
apps/web/src/components/buyer/**
docs/deck/**      EXCEPT SOURCES.md (L5)
docs/roles/R6_CONSOLES.md (create it - my first commit)
```

If a change is needed anywhere else, do NOT edit it. Append an entry to docs/BLOCKERS.md, write a local stub or a TODO(L6): comment in my own file, and keep moving. Never blocked longer than fifteen minutes.

Hard rules for you specifically:

- My first commit is docs/roles/R6_CONSOLES.md, written from T5.7, T5.8, T5.9 and T5.13 of docs/roles/R5_BAZAAR.md, so I have a task list of my own to work from.
 - Build against @mandi/contracts/fixtures. Never wait for another lane.
 - The buyer card shows renegotiation rate, not just a star rating.
 - The FPO split table shows each member's actual weight. Auditable, on screen.
 - The admin console shows coverage per mandi and the imputed share, labelled.
 - The deck starts now and gets ten minutes at every sync window, never after the freeze. Two artefacts: the official SIH Idea PPT mapped 1:1 to the template, and the 7-minute college deck.
 - Every number on a slide has provenance in docs/deck/SOURCES.md or comes off.
 - I own apps/web/src/components/ui/** as READ-ONLY. Compose, never modify.
- After each task run `npm run typecheck && npm run test`, then commit (feat(consoles): ...) and push. Commit every 30-45 minutes.

Start with writing docs/roles/R6_CONSOLES.md, then T5.7 and tell me your plan before you write any files.

07 The Cut List

You will run out of time. Every team does, and the teams that lose are the ones deciding what to drop at 02:00 on the last night, tired, with four people arguing. Decide now, while it is cheap, and write it on the whiteboard.

Drop in this order, and only in this order

#	Drop	Costs you	Say this instead
1	Disputes (`/api/disputes`, the 48-hour asymmetry guard)	Nothing on the checklist. The mechanism is still describable.	"The asymmetry guard is designed and specified - silence costs the buyer, not the farmer. Two hours of work we chose not to spend before the demo."
2	FPO console	Part of C7 - keep the farmer-side consent sheet, which is the part that matters.	"The consent flow is what we built, because that is the part farmers distrust. The FPO manager view is a table."
3	Pooling entirely	C7.	"Grade-weighted fair split with pre-consent is implemented and unit-tested in `split.ts`. The screen is next."
4	Buyer console	C8 - so demo the escrow transitions from the farmer side and the API instead.	"The escrow state machine is the same code either way. Here is the farmer's view of it."
5	E2E tests	Nothing visible. Unit tests on the domain layer are the ones that matter for the pitch.	Say nothing. Nobody asks.
6	Admin data-quality console	C-none, but you lose a genuinely distinctive screen. Drop this reluctantly.	"Coverage per mandi is computed; the console is the next thing we build."

Never drop these, under any circumstance

- `POST /api/window/recommend` - the hero endpoint. Without it there is no product, only a dashboard.
- The worst-case row on the window screen, at the same type size as the gain.
- The pledge card, labelled simulated. It is the moat and it is beat 7.
- The `NO\_ADVICE` refusal screen, driven by real band width.
- `GET /api/ledger/verify` and the live tamper demo. Fifteen seconds, and it is worth more than a paragraph about blockchain.
- Auth, and prices net of transport. C1 and C2 are the floor.
- Both decks. The college round is a presentation.

The cut list is Lane 1's to invoke, at Sunday 20:00, out loud.

Not a vote and not a discussion - a call, made by the one person whose job is integration and who can see all six branches. If nobody has authority to say "we are dropping the FPO console, stop working on it", the team drops nothing and finishes six things at 70% instead of four things at 95%. Four at 95% wins.

08 The Protocol for Six Agents in One Repository

The largest risk to this project is not missing features. It is two agents editing the same file and destroying each other's work at merge time. Everything below exists for that reason.

Sync windows - ten minutes, and non-negotiable

Friday 19:00, Saturday 12:00, Saturday 23:00, Sunday 12:00, Sunday 20:00, Monday 07:00. Round-robin, ninety seconds each, exactly three sentences: what merged since last sync, what I am on now, and what I need from whom - and that had better already be in `BLOCKERS.md`. Then Lane 1 merges in dependency order - 1, then 2 and 3, then 4, 5 and 6 - and announces "main is at X, rebase now." Everybody rebases before writing another line.

The blocker protocol - fifteen minutes, maximum

You will need something you do not own: a schema field, a contract shape, an endpoint that does not exist. Waiting stalls you. Editing their file collides. The right move, every single time:

1. **Append** to `docs/BLOCKERS.md`. Never edit or delete another entry.
2. **Stub locally** so you keep moving - a hardcoded object, a fixture from `@mandi/contracts/fixtures`, or a `TODO(L3):` comment naming the exact shape you assumed.
3. **Say it out loud** at the next sync. The file is the record; speech is the alert. Both, always.
4. When it lands, delete your stub. `grep -rn "TODO(L)"` on Sunday is your punch list.

The invariant: **no agent is blocked for more than fifteen minutes.** If you are, you are doing it wrong. Stub and move.

THE FORMAT. COPY IT EXACTLY.

```
### [L4 -> L1] CONTRACT: WindowRes needs worstCaseTotalPaise
- What I need: `worstCaseTotalPaise: number` on `WindowRes`.
- Why: the hero card shows the worst case for the whole lot, not per quintal,
  and multiplying on the client re-introduces a float.
- Blocking: T4.7, and the deck screenshot due Saturday 23:00.
- Workaround in place: computing it in the component from qtyKg. Will delete.
- Raised: Sat 09:40
```

The ten invariants, condensed

These are not style preferences. Breaking one is a bug even if the tests pass. The full text is in `CLAUDE.md` and it is binding.

#	Invariant	Why it is a rule and not a suggestion
I1	All money is integer paise. Field names end `Paise`. Format only at the render edge.	Floats lose money. A judge who spots float arithmetic in a price product is finished with you.
I2	Every read of user-owned data is scoped by the session actor. Never trust an id from the client.	Fetching another farmer's lot by id and getting data is the most common bug in hackathon marketplaces, and the technical panel will try it.
I3	`realisation_ledger` and `audit_log` are append-only. No UPDATE, no DELETE, ever.	A ledger you can silently edit is not a record. Transparency is a stated outcome of the problem statement.
I4	The model may refuse. Band too wide, return `NO_ADVICE` with a reason.	A wrong HOLD costs a farmer real money. Refusal is the ethical position and the strongest demo moment.
I5	Zero live external network calls in the demo. Everything seeded.	Venue wifi fails. It always fails.
I6	Synthetic data is labelled synthetic in the UI.	Presenting generated data as government data to a government panel is unrecoverable.
I7	No secrets in git. Only `.env.example` is committed.	A leaked secret stays in history. Deleting the line later does not remove it.
I8	No Aadhaar numbers, ever. Not hashed, not encrypted, not fake ones in the seed.	Legal exposure, trivially avoidable. Phone is the identifier.

#	Invariant	Why it is a rule and not a suggestion
19	State transitions go through the FSM in <code>`escrow.ts`</code> . Nowhere else.	Money state machines that skip states release funds without delivery.
I10	Every route validates with the Zod schema from <code>`@mandi/contracts`</code> . No hand-rolled parsing, no <code>`as any`</code> .	One validation layer, one source of truth, no drift between client and server.

Definition of done - all eight, not six of eight

1. ``npm run verify`` passes: typecheck, lint, unit tests, production build.
2. It works against **seeded** data with no external network call.
3. Zod validation on every input. Bad input returns a coded 400, not a 500.
4. Another actor's id returns 404, never their data. You have tested this by hand with two cookies and ``curl``, not by reading the code.
5. Money is paise the whole way. You have grepped your own diff for ``parseFloat``, ``Number(`` and ``.toFixed``.
6. Empty state, loading state and error state all render. Not just the happy path.
7. Marathi strings exist for anything a farmer sees. Request them from Lane 4 if missing.
8. Committed and pushed.

09 What Would Lose This For Us

Read this list once a day. Every item is a specific thing a team like ours has actually done.

Failure	How it happens	The counter-move
A feature that works in exactly one hand-typed path	You demo the path you built. The judge clicks the second thing.	Every screen gets a real empty state and a real error state. Lane 5 walks Lane 4's flow and vice versa.
A live API call in the demo	Somebody leaves a fetch in for convenience and forgets.	I5. Wifi off on stage, visibly. Grep for `fetch(` outside the ingestion job before the freeze.
A confident forecast with no interval	A single number looks cleaner on a chart, so it ships.	p10/p50/p90 or nothing. Model card visible on the forecast screen.
An unverified number on a slide	A statistic gets pasted from memory during deck-building at 02:00.	Every external figure carries provenance in `docs/deck/SOURCES.md` or it comes off the slide.
Blockchain theatre	Somebody suggests a chain would impress the panel.	A hash-chained append-only table gives tamper-evidence at zero infrastructure cost. We can explain exactly why a chain is the wrong call. Do not add one.
Feature work after the freeze	Sunday 22:00 and someone has "a quick idea".	H58 is a hard rule, not a target. Lane 1 says no.
An undeclared blocker	Somebody spends two hours writing around a missing field instead of asking.	Fifteen minutes, then `BLOCKERS.md` plus the group chat. This is the most expensive failure on the list.
Six agents, never rehearsed together	The team has never coordinated multiple Claude Code sessions in one repo before.	Friday 15:00-19:00 is a coordination dry run: everyone boots, reads, makes one trivial commit, rebases. Find the friction on Friday, not on Sunday.

10 The Next Sixty Minutes

Not the next three days. The next hour, per person, starting when you finish reading this.

Who	In the next hour
Everyone	Clone. <code>`npm install`</code> . <code>`npm run db:up`</code> . <code>`cp .env.example .env`</code> and generate <code>`AUTH_SECRET`</code> . Read <code>`CLAUDE.md`</code> end to end - all of it, it is 234 lines. Read your lane brief. Create your branch. Make one trivial commit and push it, so we know six people can push to this repo before we depend on that.
Pranay (L1)	Push the repository first - one command, section 12. Then scaffold <code>`apps/web`</code> and push it. Three people are waiting on that directory. Then <code>`npm run typecheck`</code> and tell the group what it says.
Akash (L2)	<code>`uv venv --python 3.11`</code> in <code>`services/ml`</code> . Verify LightGBM imports before you write anything else. Then start ingestion. If the environment fights you, say so in the chat within fifteen minutes.
Nikhil (L3)	Write the escrow transition table on paper first - eleven rows, three terminal states - then have your agent turn it into <code>`escrow.ts`</code> plus tests in one commit. No routes yet.
Shreya (L4)	Create <code>`messages/en.json`</code> and <code>`messages/mr.json`</code> and the language context before any screen. Then the shell. Open <code>`@mandi/contracts/fixtures.ts`</code> and read it - that file is your entire back end until Sunday.
Kartik (L5)	Read <code>`docs/03_DEMO_AND_SEED.md`</code> section 3, all of it. Then reference data, and get the warehouse rows in early - find the WDRA registry entries for the Nashik belt and record the URL and pull date in a comment on the row.
Nilesh (L6)	Write <code>`docs/roles/R6_CONSOLES.md`</code> from section 06 of this document as your first commit, so your agent has a task list to work from. Then the buyer console shell, and the deck outline with a dated screenshot punch list.

11 After the College Round

The internal submission shows 80-90%. Everything below is what closes the gap before the main SIH hackathon, and naming it honestly on a roadmap slide reads as maturity. Claiming you built it reads as a lie the moment a judge clicks.

Phase 2	Why it was out of scope for three days
Live Agmarknet and eNAM polling	We ingest offline into a seeded database, because a live call is a demo that dies on venue wifi. The ingestion script is real; the schedule is not yet.
SMS and IVR	The design is in the deck. A farmer with a feature phone is the majority of the market and this is the highest-value item on this list.
Real e-NWR and WDRA pledge execution	The arithmetic, the guardrail and the schema are built. No lender is connected, and the card says so on screen.
ONDC and eNAM transaction integration	Named in the roadmap, not claimed. Integration is a partnership, not a sprint.
A payment gateway	Escrow is simulated and labelled simulated everywhere it appears.
Native mobile app	PWA only. Installable, works on a Rs. 6,000 phone on 3G, and one codebase.
Weather and yield forecasting	Rainfall anomaly is an input feature to the price model. It is never an output. A bad weather forecast is worse than none.
The remaining consoles and disputes	Whatever the cut list took on Sunday night goes here, with an estimate next to it.

12 Pushing the Repository

Everything currently in the working tree is committed locally on `main`. The push could not be completed from this session because outbound network access to GitHub is blocked in the sandbox. Run this yourself, once, from the repository root:

RUN THIS

```
git push -u origin main
```

If it asks for credentials, use a GitHub personal access token as the password, not your account password. Then confirm all six people can clone and push before Friday 19:00 - a permissions problem discovered on Sunday costs a whole evening.

What is in that commit

Commit `11151d9`, on top of `a4e1693`: the contracts package, the Prisma schema, eleven documents, five role briefs, the seed skeleton and its deterministic PRNG, the ML contract mirror, and the workspace configuration. Only `.env.example` is committed - no secrets, per I7. Nine stale `.pyc` files were removed from tracking and `.gitignore` was extended to keep them out.

One thing to check before anyone else clones

Confirm the repository is private if you intend it to be. And confirm that `.env` is absent from `git ls-files` on every machine, not just yours - the file is gitignored, but a teammate who copies it under a different name defeats that in one command.

The one sentence to keep in front of you for three days

The binding constraint on farmer price realisation is not information - it is the ability to wait. We are not building a price dashboard. We are building a waiting product: it tells a farmer whether waiting pays, by how much, with what confidence, and then removes the liquidity and storage reasons he could not wait. Every task in this document serves that sentence. If a task does not serve it, cut it.